Journal de bord – PFE CANvisionNative

Mohamed Mootaz Ben Kraiem · ESPRIT, Information Technology · 28 octobre 2026

Informations

Ce journal retrace, toutes les deux semaines, la réalisation de mon projet de fin d'études CANvisionNative, de février à octobre 2026. Chaque entrée reprend les questions du guide : missions, apprentissages, compétences, difficultés et organisation.

Élément	Détail
Nom et prénom	Mohamed Mootaz Ben Kraiem
Spécialité	Information Technology
Organisme d'accueil	Capgemini Engineering, division Automobile de Capgemini Tunisie, équipe Battery Management Systems (BMS)
Cadre	Alternance depuis septembre 2024 : testeur QA des logiciels BMS pour BMW Group, du lundi au vendredi, et cours à ESPRIT le samedi
Projet	CANvisionNative, plateforme de diagnostic et de validation du trafic CAN des BMS, menée de février à octobre 2026
Encadrement	Mme Nadia Boulifa (ESPRIT), M. Hamda Guizani (Capgemini Engineering)
Méthode	Processus Unifié : inception, élaboration, construction en quatre itérations, transition

Du 2 au 15 février 2026 : la phase d'inception

- **Missions :** J'ai transformé les difficultés de mon travail de validation BMS en une vision de projet. J'ai identifié les acteurs (l'analyste, le véhicule ou bus CAN, le fournisseur de modèle de langage), rédigé les exigences fonctionnelles et non fonctionnelles, construit le diagramme de cas d'utilisation et listé les risques initiaux.
- **Ce que j'ai appris :** Une exigence utile se vérifie : j'ai appris à la formuler comme un cas d'utilisation et à lui donner une priorité (must, should, could). J'ai aussi appris à ordonner le travail par les risques, en plaçant au début ce qui pouvait faire échouer le

projet.

- **Compétences mobilisées :** Analyse des besoins, modélisation UML, connaissance du bus CAN acquise dans l'équipe BMS.
- **Difficultés et réponses :** J'avais trop d'idées à la fois : diagnostic, surveillance en direct, cybersécurité. J'ai tranché en faisant du diagnostic l'objectif principal, la sécurité restant une motivation secondaire.
- **Organisation :** J'ai comparé le cycle en cascade, Scrum et le Processus Unifié, puis choisi ce dernier : il convient à un développeur seul et met l'architecture et les risques au centre. J'ai planifié le projet par phases et par jalons, de M1 à M8.

Du 16 février au 1er mars : l'architecture

- **Missions :** J'ai conçu l'architecture de la plateforme : un client de bureau Windows (C#, WPF, .NET Framework 4.8) et un backend Python (FastAPI) sur la même machine, qui communiquent en HTTP. J'ai décidé qu'un seul moteur d'exécution traiterait le rejeu, la simulation et la capture en direct.
- **Ce que j'ai appris :** Le patron MVVM, où chaque écran est lié à son ViewModel par la liaison de données, et la construction d'une API REST typée avec FastAPI.
- **Compétences mobilisées :** Conception logicielle, diagrammes de composants et de déploiement, choix techniques argumentés.
- **Difficultés et réponses :** La cible .NET Framework 4.8 limite les bibliothèques et certaines fonctionnalités récentes du langage. J'ai retenu CommunityToolkit.Mvvm, compatible avec cette version, et vérifié chaque dépendance avant de l'adopter.
- **Organisation :** J'ai suivi le principe « l'architecture d'abord » : relier les deux programmes de bout en bout avant d'ajouter des fonctions. Le moteur commun réduisait déjà deux risques : l'absence de véhicule pour la capture en direct et des formats lus différemment selon le chemin.

Du 2 au 15 mars : le socle exécutable (jalon M1)

- **Missions :** J'ai terminé le jalon M1 le 13 mars : le client démarre et arrête une session en direct par l'API, le backend note un flux simulé trame par trame, une alerte remonte jusqu'au client et le tableau de bord trace le score des dernières trames. Un test de fumée vérifie toute cette chaîne. En parallèle, j'ai commencé à rassembler les jeux de données publics pour l'entraînement.
- **Ce que j'ai appris :** Une chaîne complète, même très simple, révèle tôt les problèmes d'intégration qu'un composant testé seul ne montre pas.
- **Compétences mobilisées :** Développement C# et Python, tests de bout en bout.

- **Difficultés et réponses :** Les états du moteur et ceux de l'interface se désynchronisaient. J'ai restructuré le backend pour séparer le rejeu, la télémétrie, le moteur et la visualisation, chacun avec son propre état.
- **Organisation :** J'ai découpé le jalon en petites étapes vérifiables, chacune terminée par une démonstration dans le client.

Du 16 au 29 mars : les données et les formats

- **Missions :** J'ai écrit une fonction unique qui convertit chaque format d'enregistrement en un même flux de trames (horodatage, identifiant, données) : CSV, dont celui de SavvyCAN, MF4 lu avec asammdf, Vector .asc, candump .log et PCAN .txt. J'ai rassemblé les enregistrements de CSS Electronics, de ROAD, de Car-Hacking et de EV-CANlogs.
- **Ce que j'ai appris :** Chaque outil d'acquisition a ses conventions : unités d'horodatage, colonnes, séparateurs. Une représentation commune évite que les résultats dépendent du format.
- **Compétences mobilisées :** Python, pandas, lecture de formats binaires et texte.
- **Difficultés et réponses :** La première base de données, statique, donnait des transitions d'états et des anomalies incohérentes. J'ai reconstruit le pipeline sur des enregistrements réels et dynamiques, comme le jeu Car-Hacking.
- **Organisation :** J'ai tenu un inventaire des sources, avec pour chacune le véhicule, le format et l'usage prévu, pour savoir à tout moment quelles données servaient à quoi.

Du 30 mars au 12 avril : les premiers modèles

- **Missions :** J'ai construit la première conception des modèles (jalon M3) : quatorze caractéristiques glissantes par identifiant, un K-Means qui classe l'état de conduite (stationnement, ralenti, conduite) et un Isolation Forest par état et par véhicule. Le 6 avril, j'ai commencé le moteur d'explication (jalon M4).
- **Ce que j'ai appris :** Pourquoi choisir un apprentissage non supervisé : la plupart des enregistrements n'ont aucune étiquette d'attaque, et un modèle appris sur des attaques connues reconnaît mal une attaque nouvelle. Isolation Forest isole une trame inhabituelle en peu de découpages.
- **Compétences mobilisées :** scikit-learn, ingénierie de caractéristiques, lecture d'articles scientifiques sur la détection d'intrusion CAN.
- **Difficultés et réponses :** Je ne pouvais pas mesurer une précision sans étiquettes. J'ai décidé de juger les modèles sur leur comportement face à du trafic normal, et de rester prudent sur ce que je pouvais affirmer.

- **Organisation :** J'ai fait se chevaucher deux jalons (M3 et M4), car ils touchaient des parties différentes du code.

Du 13 au 26 avril : le moteur d'explication

- **Missions :** J'ai terminé l'entraînement des premiers modèles le 24 avril et avancé le moteur d'explication. Pour chaque alerte, le backend rassemble un contexte : valeurs mesurées, scores par couche, comportement habituel de l'identifiant, schémas d'anomalie connus et cas passés similaires. Gemini rédige l'explication ; s'il ne répond pas, un modèle local Ollama prend le relais, puis un modèle de texte à base de règles.
- **Ce que j'ai appris :** Un modèle de langage explique mieux quand on lui donne des faits mesurés. J'ai aussi appris à séparer l'explication de la détection : expliquer une alerte ne change jamais son score.
- **Compétences mobilisées :** Intégration d'API de modèles de langage, construction de contexte (RAG), Python.
- **Difficultés et réponses :** Une clé absente ou une coupure réseau pouvaient laisser l'analyste sans explication. La chaîne Gemini, puis Ollama, puis règles garantit toujours un texte, avec le nom du moteur qui l'a écrit.
- **Organisation :** J'ai traité ce risque (R2) dès la conception, comme le prévoyait la liste des risques.

Du 27 avril au 10 mai : les graphiques temps réel

- **Missions :** J'ai poursuivi le moteur d'explication et commencé le 4 mai les graphiques temps réel (jalon M5) : courbe du score d'anomalie, moniteur des trames du bus et activité par identifiant.
- **Ce que j'ai appris :** Afficher des données qui arrivent en continu demande de limiter ce que l'on redessine. Le client interroge le backend à intervalle régulier et ne met à jour que ce qui a changé.
- **Compétences mobilisées :** WPF, liaison de données, programmation asynchrone en C#.
- **Difficultés et réponses :** Les tâches longues, comme une analyse ou un rejeu, ne doivent pas bloquer l'interface. Je les ai lancées en arrière-plan, le client suivant leur état par interrogation.
- **Organisation :** J'ai vérifié chaque graphique sur un enregistrement réel avant de passer au suivant.

Du 11 au 24 mai : le décodage des signaux

- **Missions :** J'ai continué les graphiques et commencé le 18 mai le décodage hors ligne et en direct (jalon M8). Les profils de véhicules indiquent le débit du bus et les fichiers DBC qui transforment les octets d'une trame en signaux physiques, comme une vitesse ou un état de charge.
- **Ce que j'ai appris :** La structure d'un fichier DBC : pour chaque message, ses signaux, leur position dans les octets, leur facteur et leur décalage. J'ai utilisé la bibliothèque cantools pour l'appliquer.
- **Compétences mobilisées :** Connaissance du bus CAN et des messages BMS, Python.
- **Difficultés et réponses :** Peu de véhicules ont un fichier DBC public, et ceux qui existent sont incomplets. J'ai décidé de ne jamais deviner un signal : sans DBC, la plateforme montre les octets bruts et les marque « non décodés ».
- **Cohérence avec le métier :** Ce travail est proche de mon quotidien de testeur BMS, où je vérifie la plausibilité de ces mêmes signaux.

Du 25 mai au 7 juin : l'enregistrement de session

- **Missions :** J'ai développé l'enregistrement de session (jalon M6) : une session simulée ou en direct est sauvegardée dans un fichier de log, avec en option une anomalie injectée et étiquetée. J'ai poursuivi le décodage en parallèle.
- **Ce que j'ai appris :** Une anomalie injectée et étiquetée donne une référence connue pour tester le pipeline, là où les enregistrements publics n'en ont pas.
- **Compétences mobilisées :** Écriture de fichiers aux formats d'échange, conception de tests.
- **Difficultés et réponses :** Un enregistrement doit pouvoir être relu par le même lecteur que les autres formats. J'ai fait en sorte que les fichiers produits passent par la fonction de lecture commune.
- **Organisation :** Avec trois jalons menés en même temps (M5, M6 et M8), j'ai tenu une liste de tâches par jalon pour ne pas perdre le fil.

Du 8 au 21 juin : l'intégration et la fin de l'itération 1

- **Missions :** J'ai intégré le client et le backend (jalon M7) et livré les jalons M3 à M8 le 22 juin. Les modèles ont été réentraînés sur plus de sept millions de trames issues de soixante-six sources. L'analyste peut désormais importer un log, l'analyser, le rejouer, expliquer ses alertes et les exporter en CSV ou en PDF.

- **Ce que j'ai appris :** Qu'une fonction terminée n'est pas une fonction validée : chaque jalon marchait en démonstration, mais rien n'avait encore été comparé à des faits connus indépendamment.
- **Compétences mobilisées :** Intégration logicielle, génération de rapports PDF avec PDFsharp.
- **Difficultés et réponses :** Les pièces développées séparément devaient fonctionner ensemble dans le client. J'ai rejoué des enregistrements réels à chaque intégration.
- **Organisation :** À la fin de l'itération, j'ai fait le bilan de son incrément et décidé que la suivante serait entièrement consacrée à la validation.

Du 22 juin au 5 juillet : la validation commence

- **Missions :** J'ai lancé la deuxième itération sur un enregistrement réel d'une heure d'un Kia EV6 (97 741 trames). J'ai d'abord audité l'interface : chaque écran, chaque bouton, des actions dans le mauvais ordre, un redimensionnement pendant un rejeu. En comparant le nombre de trames converties aux métadonnées du fichier MF4, j'ai trouvé et corrigé un défaut de conversion.
- **Ce que j'ai appris :** Une règle que j'applique depuis : une affirmation n'est acceptée qu'avec une preuve reproductible, et un défaut n'est « trouvé » que lorsqu'on sait le déclencher à volonté.
- **Compétences mobilisées :** Mon expérience de testeur QA : tests exploratoires, reproduction pas à pas, rapports de défaut.
- **Difficultés et réponses :** Il fallait un enregistrement fiable : véhicule de série, durée suffisante, profil et modèle disponibles. Celui du Kia EV6 remplissait ces conditions.
- **Cohérence avec le métier :** C'est la partie la plus proche de mon métier de testeur, appliquée cette fois à mon propre logiciel.

Du 6 au 19 juillet : l'analyse des alertes et les premières corrections

- **Missions :** J'ai attribué chacune des 15 945 alertes du Kia EV6 (16,3 % des trames) à la couche qui l'avait provoquée. Cette analyse a révélé quatre défauts, que j'ai corrigés un par un : le modèle du véhicule choisi n'était jamais utilisé, l'état de conduite était classé « stationnement » pour 99,93 % d'une heure de conduite, la cause affichée d'une alerte était fausse, et le rejeu contournait le lecteur commun.
- **Ce que j'ai appris :** Les défauts les plus coûteux sont silencieux : aucun ne faisait planter le logiciel. Un gestionnaire d'erreurs trop large renvoyait toujours « stationnement » et masquait une erreur de 11 caractéristiques au lieu de 14.
- **Compétences mobilisées :** Analyse de cause racine, lecture de code, comparaison avec des faits connus, comme un véhicule en train de rouler.

- **Difficultés et réponses :** Il était tentant de baisser un seuil pour avoir de meilleurs chiffres. J'ai refusé : chaque correction devait s'appuyer sur une cause trouvée dans le code.
- **Organisation :** Pour chaque défaut, j'ai suivi le même modèle : problème, investigation, cause, correction, preuve, résultat.

Du 20 juillet au 2 août : les corrections 5 et 6 et la nouvelle validation

- **Missions :** En suivant les cas d'utilisation dans le client, j'ai trouvé deux défauts de plus : un minuteur resté arrêté après un import, et un cache qui pouvait renvoyer l'explication d'une autre alerte. Je les ai reproduits avec deux enregistrements de véhicules différents (Kia EV6 et Renault Clio), puis corrigés. La nouvelle validation a donné 11 971 alertes au lieu de 15 945, un débit passé de 143 à environ 195 trames par seconde, et 99,92 % de l'heure classée en conduite.
- **Ce que j'ai appris :** Tester le backend seul ne suffit pas : ces deux défauts n'apparaissaient qu'en passant par l'application.
- **Compétences mobilisées :** Tests de bout en bout, mesures avant et après, présentation de résultats par des graphiques.
- **Difficultés et réponses :** Prouver qu'une explication appartient bien à la bonne alerte. J'ai comparé l'identifiant et le score de chaque alerte avec son explication, sur les deux jeux de données.
- **Organisation :** Une correction à la fois, chacune vérifiée à nouveau avant la suivante, puis une exécution complète du pipeline corrigé.

Du 10 au 23 août : la couche matérielle

- **Missions :** J'ai commencé la troisième itération (jalon M2) : une couche matérielle sur python-can, qui accepte les adaptateurs slcan, PEAK PCAN, SocketCAN et un bus virtuel pour les tests. Chaque trame reçue passe par le même moteur que le rejeu. Faute de véhicule, j'ai conçu un banc de test : deux cartes Arduino Uno avec des modules CAN MCP2515, reliées par un bus terminé par des résistances de 120 Ω, à 500 kbit/s.
- **Ce que j'ai appris :** Les bases de la couche physique CAN : la terminaison, le débit commun à tous les nœuds, et le rôle d'un adaptateur entre le bus et le PC.
- **Compétences mobilisées :** Électronique de base, python-can, conception d'un environnement de test.
- **Difficultés et réponses :** Aucun véhicule compatible n'était disponible, c'était le risque R1. Comme le moteur est commun à tous les chemins, un banc de test suffit à vérifier la partie propre à la capture en direct : le transport des trames.

- **Cohérence avec le métier :** Ce travail m'a rapproché du matériel que valident les équipes BMS.

Du 24 août au 6 septembre : les micrologiciels du banc

- **Missions :** J'ai programmé les deux cartes. La carte passerelle, reliée au PC par USB, parle le protocole slcan attendu par python-can. La carte simulateur envoie une trame de vitesse (60 km/h) toutes les 20 ms, une trame d'état de charge (78 %) toutes les 200 ms et, toutes les vingt secondes, une trame sur un identifiant qui ne devrait pas exister (0x666).
- **Ce que j'ai appris :** La programmation Arduino en C++ et le protocole série slcan, qui transforme chaque trame CAN en une ligne de texte.
- **Compétences mobilisées :** Programmation embarquée, communication série, débogage matériel.
- **Difficultés et réponses :** Un banc qui ne marche pas peut venir du câblage, de la carte ou du logiciel. J'ai vérifié par étapes : d'abord un court script python-can qui lit le bus, puis seulement l'application.
- **Organisation :** J'ai documenté le câblage et les messages envoyés pour pouvoir rejouer le test à l'identique.

Du 7 au 20 septembre : le test du banc et la reconstruction des modèles

- **Missions :** Le 12 septembre, j'ai connecté le banc depuis l'application. Ce test a révélé deux défauts, que j'ai corrigés. Une seule ligne série mal formée fermait la session, après 16 304 trames : elle est désormais ignorée et journalisée. Des données anciennes restaient affichées après le débranchement : le lien est maintenant déclaré perdu après cinq secondes sans trame. Le 14 septembre, j'ai lancé la quatrième itération : un Isolation Forest par identifiant CAN, neuf caractéristiques, un seuil propre à chaque modèle et un manifeste qui trace comment chaque modèle a été construit.
- **Ce que j'ai appris :** Pour juger un modèle sur des données nouvelles, il faut séparer les données par enregistrement, et non par trame : deux trames d'un même enregistrement se ressemblent trop.
- **Compétences mobilisées :** Robustesse d'une liaison série, apprentissage automatique, reproductibilité (empreintes SHA-256, versions du code).
- **Difficultés et réponses :** La validation de juillet avait montré que la première conception des modèles était fragile. Plutôt que de la réparer encore, j'ai choisi de la reconstruire entièrement.

- **Organisation :** Pour éviter de mélanger ancien et nouveau, j'ai mis en place un dossier de préparation, puis une étape de promotion vers la production.

Du 21 septembre au 4 octobre : l'évaluation et la transition

- **Missions :** J'ai évalué chaque modèle sur un enregistrement complet qu'il n'avait jamais vu. Quatre modèles sont entrés en production (296 modèles d'identifiant, testés sur 1 130 500 trames), avec des taux de trames signalées de 0,60 % à 6,09 %. Le modèle du MG ZS EV a été écarté : il signalait 43,60 % des trames. J'ai aussi comparé Isolation Forest à LOF et au One Class SVM. La transition a commencé : tests automatisés (318 côté backend, 86 côté client), guide d'installation, profil et fichier DBC complets pour le banc, nouveau rapport PDF.
- **Ce que j'ai appris :** Sans étiquettes d'attaque, on mesure un taux de trames signalées, pas un taux de détection. Je dois le dire clairement plutôt que de présenter un chiffre trompeur.
- **Compétences mobilisées :** Évaluation de modèles, analyse de résultats, rédaction technique.
- **Difficultés et réponses :** Le taux du jeu ROAD (6,09 %) dépassait l'objectif. J'en ai cherché la cause, un identifiant dont les données changent seulement sur l'enregistrement de test, sans modifier le seuil après coup.
- **Organisation :** La transition a commencé le 21 septembre, en chevauchement avec la fin de l'itération 4.

Du 5 au 18 octobre : le rapport, la présentation et la démonstration

- **Missions :** J'ai rédigé le rapport et, sur les conseils de mon encadrante, je l'ai restructuré le 15 octobre selon les phases du Processus Unifié : inception, élaboration, construction en quatre itérations, transition. J'ai préparé la présentation, une vidéo de démonstration commentée et un enregistrement de l'application en direct sur le banc de test : vitesse de 60 km/h décodée, puis alerte sur l'identifiant 0x666, expliquée par le module d'explication.
- **Ce que j'ai appris :** Organiser un rapport selon la méthode suivie le rend plus lisible : chaque chapitre correspond à une phase du travail. J'ai aussi appris à présenter un travail technique à un jury en peu de temps.
- **Compétences mobilisées :** Rédaction en LaTeX, communication orale, préparation d'une démonstration.
- **Difficultés et réponses :** Lors du premier enregistrement en direct, l'application était connectée au simulateur virtuel avec le profil d'un autre véhicule. J'ai vérifié la configuration (slcan, port COM5, 500 kbit/s, profil du banc), redémarré le backend,

puis refait l'enregistrement.

- **Organisation :** Avec des délais très courts, j'ai figé l'application et me suis concentré uniquement sur le rapport et la présentation.

Du 19 au 28 octobre : les corrections du rapport et le dépôt de la version 1

- **Missions :** J'ai appliqué les retours de mon encadrante en deux tours. Le premier traitait 58 commentaires : figures placées juste après le texte qui les cite, titres jamais séparés de leur contenu, page de signatures, nouvelles sections « Diagrammes de classes » et « Planification des itérations », diagrammes de séquence aux messages numérotés, listes à puces, conclusion générale sans titres. Le second traitait huit remarques : introductions de chapitre non numérotées, numérotation limitée à trois niveaux, captures d'écran du chapitre 5 refaites en thème clair. J'ai ensuite préparé le dépôt de la version 1.
- **Ce que j'ai appris :** Les normes de présentation d'un rapport comptent autant que son contenu pour le lecteur : numérotation, placement des figures, phrases d'introduction avant chaque tableau.
- **Compétences mobilisées :** Mise en page LaTeX, modélisation UML (objets boundary, control et entity dans les diagrammes de séquence), relecture.
- **Difficultés et réponses :** Certaines corrections en entraînaient d'autres : déplacer une figure décalait les pages suivantes. Après chaque changement, j'ai vérifié toutes les pages.
- **Organisation :** J'ai traité les commentaires dans l'ordre du rapport et coché chacun une fois vérifié dans le PDF final.